Weld Study Protocol (Draft v0.1)

An empirical study of how an integrated developer workspace affects productivity, cognitive load, and subjective experience compared to a multi-tool baseline.

---

1. Background and Motivation

Modern software development is fragmented across many specialized tools: an issue tracker (Linear, Jira), a chat platform (Slack, Discord), a code host (GitHub), a secrets manager (1Password/Doppler), and a time tracker — each with its own window, account, notification stream, and mental model. Sergeyuk et al.'s 2026 longitudinal study of 800 developers found that AI-assisted developers showed a statistically significant *increase* in IDE window activations (a proxy for context switching) of +6.4/month, contrary to the common assumption that integrated tools would reduce switching. Their telemetry also revealed a consistent gap between self-reported productivity gains and what behavioral logs actually showed, demonstrating that perception and behavior must both be measured.

Tang et al. (2024) showed that fine-grained IDE telemetry combined with cognitive workload self-report (NASA-TLX) and semi-structured interviews can reliably surface workflow differences that subjective measures alone miss. Their lab study with 28 participants validated this mixed-methods design at a scale realistic for short-timeline academic research.

Sergeyuk, Koshchenko et al. (2024) interviewed 35 developers about unmet needs in IDE-based AI assistants and found that *internal system integration* — the cohesion between tools within a development environment — was consistently named as a top need across adopters, churners, and non-users. However, no controlled study has measured whether a tightly integrated workspace actually delivers the cognitive and productivity benefits developers expect from it.

Weld is a cross-platform developer workspace that combines task management, team chat, GitHub integration, and team functionality into a single application sharing one context model (a task can reference a chat message, a chat message can reference a PR, a project ties them all together). This study uses Weld as a research instrument to test whether tool integration produces measurable improvements over a comparable multi-tool baseline.

2. Research Questions and Hypotheses

RQ1. Does using an integrated workspace (Weld) reduce time-on-task and increase task completion rate compared to a multi-tool baseline (Linear + Slack + GitHub) on equivalent software development tasks?

RQ2. Does using an integrated workspace reduce self-reported cognitive load (NASA-TLX) compared to the multi-tool baseline?

RQ3. How do developers describe the subjective experience of working in an integrated vs. multi-tool workflow, and what specific moments of integration (or fragmentation) do they identify as most impactful?

Hypotheses.

- **H1 (productivity).** Participants will complete the task set faster in the Weld condition than in the multi-tool condition.
- **H2 (cognitive load).** Participants will report lower NASA-TLX scores in the Weld condition, particularly on the Mental Demand and Effort subscales.
- **H3 (subjective experience).** Participants will rate the Weld condition higher on flow-state and satisfaction items, and qualitative analysis will surface integration-specific themes (e.g., reduced re-explanation of context, fewer interruptions).
- **H0 (alternative).** Following Sergeyuk et al.'s 2026 finding that perceived productivity often diverges from behavioral data, we also predict a measurable gap between subjective satisfaction with Weld and objective time-on-task — i.e., participants may *prefer* Weld even on tasks where they were not actually faster.

The last hypothesis is the most theoretically interesting and the most publishable angle, because it positions the study as a contribution to the broader literature on perception–behavior gaps in developer tooling, not just a product evaluation.

3. Study Design

Design type. Within-subjects, counterbalanced, controlled lab study with two conditions:

- **Condition A — Integrated:** Participant completes the task set using Weld.
- **Condition B — Baseline:** Participant completes an equivalent task set using Linear (or GitHub Projects) + Slack (or Discord) + GitHub.

Each participant completes both conditions in two separate sessions roughly one week apart. Order is counterbalanced — half start with Weld, half start with the baseline — to control for learning effects. Task sets are matched in difficulty and structure but not identical, to prevent participants from solving the same problem twice.

Why within-subjects. With a target of ~16–24 participants (realistic for a 1–2 month timeline with two researchers), a within-subjects design gives substantially more statistical power than between-subjects for the same N, because each participant serves as their own control. This is the same logic Tang et al. used in their 28-participant study.

Why counterbalanced. Order effects are a real threat — participants will be more familiar with the tools and tasks by their second session. Counterbalancing distributes this effect evenly across conditions.

Why two sessions. A single session would conflate tool-learning with workflow effects. By splitting across sessions and giving participants a brief pre-task practice period with each toolset, we measure steady-state workflow rather than first-impression friction.

4. Participants

Target N. 20 participants (10 starting with Weld, 10 starting with the baseline). This is a defensible sample size for a within-subjects lab study at venues like CHI Late-Breaking Work or an ICSE workshop — Tang et al. published at the main ICSE NIER track with 28; Sergeyuk et al.'s HAX paper interviewed 35.

Recruitment. A mix of university students (undergraduate and graduate CS) recruited through course mailing lists and professional developers recruited through the researcher's network and the lab's professional contacts. Inclusion criteria:

- At least 1 year of programming experience in any language.
- Familiarity with at least one issue tracker (Linear, Jira, GitHub Issues, Trello) and one team chat (Slack, Discord, Teams).
- Familiarity with Git and GitHub.

Exclusion criteria. Inability to attend two sessions one week apart.

5. Procedure

Each session is **~75 minutes** structured as follows:

Time	**Activity**
0–5 min	Welcome, consent reminder, eye-tracker calibration (if available)
5–15 min	Brief tutorial on the toolset for this session (Weld or the baseline trio)
15–60 min	Task set (3 tasks, ~15 min each, with a 30s break between)
60–65 min	NASA-TLX questionnaire

Time	Activity
65–75 min	Short semi-structured interview

Session 1 ends with scheduling Session 2. **Session 2** ends with the post-study interview covering both conditions and a final preference question.

6. Tasks

The tasks are designed to require the participant to use multiple modules of the workspace — exactly the situation where integration should pay off (or not). Each task is scoped to ~15 minutes for a developer with average experience. The exact tasks will be developed and pilot-tested before the main study; the protocol commits to this structure:

Task 1 — Triage and respond. Participants are given a small project with 5 open issues and 12 unread chat messages. They must (a) read the messages, (b) identify which issues are blocked vs. ready to work on, (c) post a status update in chat that references the relevant issues, and (d) reassign two issues based on team availability information embedded in the chat history.

Task 2 — Code-and-link. Participants are given a specific issue. They make a small code change in a provided Git repository, open a pull request, link the PR to the issue, and post a chat message in the project channel announcing the PR.

Task 3 — Cross-reference under interruption. The participant works on a coding task while receiving scripted chat messages from the experimenter's confederate account at fixed intervals (60s, 180s, 360s into the task). Each message asks a question that requires the participant to look up information from the issue tracker or the codebase. This task is the "stress test" for context-switching cost.

The same three tasks (with different content) are used in both conditions.

7. Measures

Behavioral (objective).

- *Time-on-task.* Wall-clock time from task start to participant signaling completion.
- *Task completion rate.* Binary success per task, plus partial-credit rubric on subcomponents.
- *Application/window switches.* Counted from screen recording or OS-level activity logs.
- *Cross-tool references.* In the baseline condition, count manual context-pastes (URL copy-pastes, issue-number mentions in chat, etc.). In Weld, count uses of native cross-references.

Self-report (subjective).

- *NASA-TLX,* administered after each task and after each full session. Six subscales: Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, Frustration.
- *Flow Short Scale* (Rheinberg et al., 13 items) administered at end of each session.
- *Single-item preference and satisfaction questions* at end of Session 2.

Qualitative.

- *Semi-structured interview after each session* (~10 min) on what felt smooth, what felt friction-y, and specific moments of integration or fragmentation.
- *Final interview after Session 2* (~15 min) on direct comparison and overall preference.

8. Data Analysis Plan

Quantitative. Paired t-tests (or Wilcoxon signed-rank if non-normal) for within-subject comparisons on time-on-task, NASA-TLX subscales, and switch counts. Report effect sizes (Cohen's d) and confidence intervals, not just p-values. Following best practice in HCI, do not over-rely on null-hypothesis significance testing — focus the discussion on effect sizes and the perception–behavior gap.

Qualitative. Thematic analysis of interview transcripts following Braun & Clarke (2006), the same approach used in Sergeyuk, Koshchenko et al. (2024). Two coders, independent open coding on the first 4–6 transcripts, codebook consolidation, then full pass with inter-rater reliability check on a 20% sample.

Triangulation. The most novel contribution will be cross-referencing behavioral metrics against self-report — specifically, identifying participants who reported preferring one condition but were objectively faster in the other, and using interview data to understand why. This is the methodological move that distinguishes mixed-methods publications at CHI from straightforward A/B comparisons.

9. Risks, Limitations, and Threats to Validity

Construct validity. Time-on-task is a coarse proxy for productivity — it captures speed but not code quality or downstream debugging cost. We mitigate this by triangulating with self-report and qualitative data and by being explicit in the writeup that we measure *task completion under controlled conditions,* not real-world productivity.

Internal validity. Tool familiarity is the largest confound. Most participants will already know Slack and GitHub but will be brand-new to Weld. The brief tutorial period helps, but cannot fully eliminate this effect. We address it by (a) reporting tool-familiarity as a covariate, (b) including a "first impressions vs. settled use" question in the post-study interview, and (c) noting it explicitly as a limitation.

External validity. Two short sessions in a lab cannot capture the long-term effects of using an integrated workspace in real work — exactly the gap Sergeyuk et al.'s longitudinal study filled. We frame our contribution as complementary: a controlled snapshot of cognitive and productivity differences during representative tasks, which suggests directions for future longitudinal field research.

Researcher bias. The first author is the developer of Weld, which is a significant conflict of interest. Mitigations: (a) the professor (second author) is not involved in Weld's development and conducts a portion of the sessions; (b) analysis is pre-registered before data collection where possible; (c) the bias and its mitigation are disclosed in the paper.

10. Ethics and IRB

This study involves human participants and therefore requires IRB review at most US institutions. Concrete actions for this week:

1. Email your professor to confirm they will be co-PI or PI of record on the IRB application.
2. Identify your institution's IRB submission portal and form templates.
3. Most CS lab studies at this scope qualify for **exempt** or **expedited** review (Category 2 or 3 under the Common Rule), which typically clears in 1–4 weeks. Full board review is unlikely to be required.
4. Required documents you'll need to submit (these are the next deliverables after this protocol):
    - Consent form
    - Recruitment materials (email template, flyer)
    - Pre-study screening questionnaire
    - Task scripts and instructions
    - Interview script
    - Survey instruments (NASA-TLX, Flow Short Scale — both freely usable for research)
    - Data management plan

Privacy. Screen recordings and audio transcripts contain potentially identifying content. Data will be stored on encrypted institutional storage, accessible only to the research team, with personally identifiable information stripped from quotes before publication. Participants will be assigned anonymized IDs (P01–P20).

11. Timeline

A realistic 8-week plan, assuming IRB clears in week 3:

Week	Activity
1	Finalize protocol with professor; submit IRB application; identify pilot participants
2	Build task materials (issues, codebase, chat seed messages); set up data collection infrastructure
3	IRB approval expected; run 2 pilot sessions; iterate on tasks and timing
4–6	Main data collection (20 participants × 2 sessions = 40 sessions; aim for 6–8 sessions/week)
7	Transcription and quantitative analysis
8	Qualitative coding, writeup, draft paper

If IRB is slow, the most compressible items are pilot iteration and writeup; the data collection window is the hardest constraint.

12. Deliverables

If the study is run as designed, expected outputs:

1. A short paper suitable for **CHI Late-Breaking Work, CSCW Posters, ICSE NIER**, or a focused venue like **PLATEAU** or **VL/HCC** — these all accept 4–8 page submissions reporting smaller-scale empirical work and are well-matched to the timeline.
2. A longer-form paper for **CHI**, **CSCW**, or **ICSE** if results warrant — this would likely require expanding either N or the longitudinal window in a follow-up study.
3. An open dataset of anonymized session metrics and the task materials, published alongside the paper to support replication.

13. Open Questions for Discussion with Professor

These are the decisions I want to surface explicitly rather than make unilaterally:

- **Eye tracking.** Tang et al. used it effectively, but it adds significant infrastructure cost. Worth it for this study, or substitute with screen recording + IDE telemetry only?
- **Baseline toolset.** Linear vs. GitHub Projects vs. Jira? Slack vs. Discord vs. Teams? The choice affects how familiar participants are and how strong the baseline is. Recommendation: pick the trio most participants already use (likely GitHub Issues + Slack + GitHub itself) to give the baseline the strongest possible footing.

- **Conflict-of-interest framing.** How aggressively should we disclose and mitigate the researcher-developer overlap? Some venues will want a stronger pre-registration; others accept disclosure alone.
- **Pre-registration.** OSF pre-registration is increasingly expected for empirical HCI work and strengthens the publication case. Worth the extra week, or skip for now?
- **Pilot scope.** 2 pilots vs. 4 pilots? More pilots de-risk task design but eat into the data collection window.

This protocol is a v0.1 draft. Next deliverables, once design is locked: consent form, recruitment materials, screening questionnaire, task materials, and interview scripts.

May 19, 2026

trying to talk to team member: communication. use email/discord. jira-board. trello. Not having to switch applications . have everything in one place. no lost of context. multiple communications. have references.

within MTU, senior design. user interface design. enterprises (course).